Ders 21 Genişletilmiş Notu: Klasik Senkronizasyon Problemleri, Monitörler ve CPU Zamanlamaya Giriş: FCFS, SJF, SRTF ve Öncelikli Zamanlama

Ders 21 Genişletilmiş Notu: Klasik Senkronizasyon Problemleri, Monitörler ve CPU Zamanlamaya Giriş: FCFS, SJF, SRTF ve Öncelikli Zamanlama

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Klasik Senkronizasyon Problemleri
- 2. Sınırlı Tampon Problemi
- 3. Okuyucu-Yazıcı Problemi
- 4. Yemek Yiyen Filozoflar Problemi
- 5. Semafor Kullanım Hataları
- 6. Monitörler
- 7. Monitörle Filozoflar Çözümü
- 8. Monitörde Bekleyen Süreçlerin Sırası ve Kullanım Hataları
- 9. İşletim Sistemlerinde Senkronizasyon Örnekleri
- 10. CPU Zamanlamaya Giriş
- 11. CPU Zamanlayıcı, Kesintili ve Kesintisiz Zamanlama
- 12. Dağıtıcı ve Dağıtım Gecikmesi
- 13. Zamanlama Ölçütleri
- 14. İlk Gelen İlk Hizmet Alır (FCFS)
- 15. En Kısa İş Önce (SJF) ve Sonraki Patlamanın Tahmini
- 16. En Kısa Kalan Süre Önce (SRTF)
- 17. Öncelikli Zamanlama ve Yaşlandırma
- Örnek Sorular
- Dersin Toparlaması

1. Klasik Senkronizasyon Problemleri

Ders, bir önceki haftada kalınan slayttan (31. slayt) devam etti. Slaytlar Silberschatz, Galvin ve Gagne'nin *Operating System Concepts* kitabının 5. bölümünün sonu ile 6. bölümünün başına aitti.

Hoca senkronizasyon konusunun başında söylediği bir şeyi hatırlattı: Bir bilgisayar sisteminde karşılaşılan bazı temel senkronizasyon problemleri vardır. Bunlar isimlendirilip genel hâle getirilerek incelenir. Bu derste üç klasik problem ele alındı:

- **Sınırlı tampon (bounded buffer) problemi:** Konunun başından beri konuşulan üretici-tüketici (producer-consumer) problemidir. Sınırlı bir alana bir üretici bir şey üretip koyar, bir tüketici oradan alıp tüketir.
- **Okuyucu-yazıcı (readers-writers) problemi:** En basit ve net hâli veri tabanlarında görülür. Bir veri tabanına aynı anda okuma ya da yazma yönünde erişmeye çalışan birden fazla istemci olur. Bu erişimlerin nasıl sıralanacağı her zaman bir tasarım problemidir.

- **Yemek yiyen filozoflar (dining philosophers) problemi:** Hocaya göre metaforu biraz fantastik olsa da, pratikte çokça karşılaşılan bir durumu anlatır.

2. Sınırlı Tampon Problemi

Hoca, sınırlı tampon probleminin teknik olarak üç semaforla "tıkır tıkır" çözüldüğünü söyledi. Bu semaforlardan ikisi sayan semafor (counting semaphore), biri ikili semafor (binary semaphore) olmak zorundadır. İkili semaforun değeri 0 ya da 1'dir. Sayan semaforun değeri ise 0 ve yukarısıdır.

Elimizde n elemanlık bir tampon olsun. Üç semafor şunlardır:

- `mutex`: Tampon üzerindeki kritik işi korumak için kullanılan ikili semafordur. Başlangıç değeri 1'dir.
- `full`: Dolu yerlerin sayısını gösterir. Başlangıçta hiçbir yer dolu olmadığı için 0'a ilklendirilir.
- `empty`: Boş yerlerin sayısını gösterir. Başlangıçta n yerin hepsi boş olduğu için n'e ilklendirilir.

2.1 Üretici

Üretici bir şey üretir ve ürettiği öğeyi tamponda uygun yere koyar. Bunu yapabilmesi için en az bir boş yer olması gerekir. Bu yüzden önce `empty` üzerinde bekler, yani boş yer sayısının sıfırdan büyük olmasını bekler. Boş yer varsa tampona yazacaktır. Tampon `mutex` ile korunduğu için ardından `mutex`'i bekler. Yazma bitince kritik bölgeden çıkar ve `mutex`'e `signal` yapar.

Hoca burada dikkat çekti: Üretici `wait` işlemini `empty` üzerinde yaptı, ama `signal`'ı `full` üzerinde yapar. Çünkü `empty` ile boşalmayı bekleriz, `full` ile de bir yeri doldurduğumuzu bildiririz. `full` 0 ile başlamışsa, üretici ürettiğini koyup `signal(full)` dediğinde `full` 1 olur.

2.2 Tüketici

Tüketicide tam tersi bir yapı beklenir. Tüketicinin bir şey tüketebilmesi için bir doluluk olması gerekir. Hocanın benzetmesiyle, olmayan parayı harcayamayacağınız gibi, ortada olmayan şeyi de tüketemezsiniz. Doluluk oluştuysa `wait(full)` ile devam edilir. Sonra tampona yine `mutex` üzerinden erişilir. Böylece herhangi bir anda üretici ya da tüketicilerden yalnız biri tampona erişir. Bir dolu yer tüketildiğine göre boş yer sayısı bir artmıştır, yani sonunda `signal(empty)` yapılır.

```
/* ortak: semaphore mutex = 1, full = 0, empty = n; */

/* üretici */
while (true) {
    /* bir öğe üret */
    wait(empty);
    wait(mutex);
    /* öğeyi tampona ekle */
    signal(mutex);
    signal(full);
}

/* tüketici */
while (true) {
    wait(full);
    wait(mutex);
    /* tampondan bir öğe çıkar */
    signal(mutex);
    signal(empty);
    /* öğeyi tüket */
}
```

Bu yapı, POSIX `sem_t` semaforlarıyla yazılıp `gcc -Wall -pthread` ile derlenerek denendi. 5 elemanlı tamponda bir üretici ve bir tüketici 10000 öğeyi eksiksiz aktardı.

3. Okuyucu-Yazıcı Problemi

Hoca okuyucu-yazıcı probleminin genel bir problem olduğunu, farklı versiyonları ve varyasyonları bulunduğunu söyledi. Hangisinin kullanılacağı problemin nerede ve nasıl geçtiğine bağlıdır.

Normal şartlarda elimizde bir grup yazıcı (writer) vardır. Yazıcıların veriye hem okuma hem yazma yönünde erişimi vardır. Öbür tarafta yalnız okuma yönünde erişimi olan okuyucular (reader) vardır. Okuma konusunda daha önce konuşulmuştu: Bir bilgiyi on, yüz ya da bin kez okusanız bilgi değişmez. Okuma eş güçlü (idempotent) bir işlemdir. Bu yüzden okuyucular arasında belli bir rahatlık vardır. Birden fazla okuyucu aynı anda okuyabilir.

3.1 Birinci ve İkinci Varyasyon

- **Birinci okuyucu-yazıcı problemi (first readers-writers problem):** Hocaya göre en yaygın ve en çok karşılaşılan varyasyondur. Okuyucular önceliklidir. Okumaya hazır olan bir okuyucu bekletilmeden veriye erişebilmelidir. Tek istisna, daha önce kilidi almış ve kritik bölgeye girmiş bir yazıcının olmasıdır. Onun işi biter bitmez okuyucu devreye girmelidir.
- **İkinci okuyucu-yazıcı problemi:** Bunun tersidir. Yazıcılar önceliklidir. Hazır olan bir yazıcı bekletilmeden doğrudan yazabilmelidir. Yine kilidi almış ve işlem yapmakta olan bir okuyucu varsa, onun işi bitince yazıcı devreye girmelidir.

Kitapta önce bir çözüm önerilip sonra bunun nasıl daha iyi olabileceği tartışılmıştır. Hoca önce genel çözümü inceleyip sonra detaylara geçti.

3.2 Okuyucu-Yazıcı Kilidi ve Sorumluluk

Hoca, bazı sistemlerin okuyucu-yazıcı problemi için doğrudan bir okuyucu-yazıcı kilidi (reader-writer lock) sunduğunu söyledi. Böylece problem çözülmüş hâliyle önünüze konur.

Ama bir şey unutulmamalıdır. Az sonra konuşulacak monitör yapısı da sonuçta soyut (abstract) bir yapıdır. Daha kolay kullanım için sunulur. Ne kadar soyut olursa olsun, hatalı kullanıldığında semaforda ya da okuyucu-yazıcı kilidinde olan problemlerle yine karşılaşılır. Açlık (starvation) da olabilir, kilitlenme (deadlock) da olabilir, başka şeyler de olabilir.

> **Hocanın vurgusu:** Senkronizasyon mekanizmasını sağlamak işletim sisteminin sorumluluğudur. Bu mekanizmayı düzgün kullanmak ise uygulama geliştiricinin sorumluluğudur.

3.3 Birinci Varyasyonun Çözümü

Elimizde ortak bir veri kümesi (data set) vardır. Çözümde üç ortak öğe kullanılır:

- `rw_mutex`: Hem okuyucu hem yazıcı için ortak olarak kullanılan semafordur. Veri kümesine erişimi korur.
- `read_count`: O anda okuma yapmakta olan süreçlerin sayısını tutan değişkendir.
- `mutex`: Yalnız `read_count` değişkenine erişimi kontrol eden semafordur.

Yazıcının işi basittir. Ortak veriye erişeceği yer için `rw_mutex`'i ele geçirir, yazar ve bitirince `signal` verip çıkar.

İşin karmaşıklaştığı yer okuyucudur. Okuyucu önce `mutex` ile `read_count`'a erişimi korur ve `read_count`'u bir artırır. `read_count` 1 ise, yani ilk okuyucu ise, `rw_mutex`'i bekleyerek yazıcıları dışarıda tutar. Hoca burada kodun girintilerine dikkat çekti: Okuyucu okumaya başlamadan önce `mutex`'i hemen `signal`'lar. Böylece bir başka okuyucu da girip `read_count`'u artırabilir. Çıkarken ters sıra izlenir. `read_count` yine değişeceği için `mutex` beklenir, `read_count` azaltılır. Değer sıfır olduysa, yani son okuyucu çıkıyorsa, `rw_mutex` serbest bırakılır. En sonda `mutex` `signal`'lanır.

```
/* ortak: semaphore rw_mutex = 1, mutex = 1; int read_count = 0; */

/* yazıcı */
while (true) {
    wait(rw_mutex);
    /* yazma */
    signal(rw_mutex);
}

/* okuyucu */
while (true) {
    wait(mutex);
    read_count++;
    if (read_count == 1)
        wait(rw_mutex);    /* ilk okuyucu yazıcıları dışarıda tutar */
    signal(mutex);
    /* okuma */
    wait(mutex);
    read_count--;
    if (read_count == 0)
        signal(rw_mutex);  /* son okuyucu kilidi bırakır */
    signal(mutex);
}
```

Bu kod da POSIX semaforlarıyla iki yazıcı ve dört okuyucu iş parçacığıyla derlenip çalıştırıldı. Program kilitlenmeden bitti ve sonunda `read_count` 0'a döndü.

3.4 Açlık Sorunu

Hoca bu slaytın varyasyonlar yüzünden çok önemli olmadığını söyledi. Önemli olan şudur: Bu çözüm kilitlenmeyi önler, ama açlığı önlemez.

Okuyucular öncelikli olduğu için okuma yapmak isteyen biri olduğu sürece yazıcı kritik bölgeye sokulmaz. Hocanın örneğinde bir veri tabanı sistemine okuma yönünde sürekli işlem (transaction) gelir ve bunlar hep önceliklendirilir. Yazmak için bekleyen süreç bir türlü yazamaz ve sonunda "açlıktan pert olur". Bunu önlemek için daha farklı, daha az ilkel bir çözüm gerekir. Bunu her seferinde yeniden tasarlamak zor olduğu için sistemler okuyucu-yazıcı kilitlerini hazır olarak sunabilir.

4. Yemek Yiyen Filozoflar Problemi

4.1 Problemin Tanımı

Hoca, problemin bilgisayarla ilgisinin ilk bakışta anlaşılmayabileceğini, ama yapı anlatıldıkça netleşeceğini söyledi.

Orijinal problemde beş filozof vardır. Bu beş filozof bir masaya oturur. Ortada bir pirinç kâsesi vardır. Filozoflar ya düşünür ya yer. Düşünmedikleri zaman acıkırlar ve yemek isterler. Japon usulü yemek çubuğuyla (chopstick) yerler. Ama beş kişilik masada toplam beş çubuk vardır. Her iki filozofun arasında bir çubuk bulunur.

Bir filozofun pilavdan yiyebilmesi için sağındaki ve solundaki çubukları alması gerekir. Herkes yalnız sağındaki ve solundaki çubukları kullanabilir. Masanın öbür ucuna uzanıp başkasının çubuğunu alamaz. Dolayısıyla bir filozof yerken sağındaki ve solundaki komşusu yiyemez. Problem, kimin ne zaman ve nasıl yiyeceğinin düzenlenmesidir.

4.2 Basit Semafor Çözümü

En basit çözümde çubuklar bir semafor dizisi olarak tutulur. `chopstick[0]`'dan `chopstick[4]`'e kadar her eleman bir çubuğu gösterir ve hepsi 1 ile başlar. Örneğin 0 numaralı filozof önce 0 numaralı çubuğu, sonra 1 numaralı çubuğu bekler. İkisini alırsa yer. Sonra iki çubuğu da bırakır.

```
/* semaphore chopstick[5]; hepsi 1 */
while (true) {
    wait(chopstick[i]);
    wait(chopstick[(i + 1) % 5]);
    /* ye */
    signal(chopstick[i]);
    signal(chopstick[(i + 1) % 5]);
    /* düşün */
}
```

4.3 Kilitlenme

Hoca sınıfa bu yapıda bir problem olup olmadığını sordu. Bir öğrencinin cevabından yola çıkarak sorunu açıkladı. Bir filozof ilk çubuğu aldıktan sonra ikincisini alamayabilir.

Senaryo şöyledir. 0 numaralı süreç `wait` deyip 0 numaralı çubuğu alır. Tam o sırada bağlam değişimi (context switch) olur. 1 numaralı süreç gelir, 1 numaralı çubuğu alır. Aynı şey 2, 3 ve 4 için de olur. Sonunda herkes solundaki çubuğu almıştır. Hiçbirinin sağında boşta çubuk kalmamıştır ve herkes birbirini bekler.

Hoca bu durumu, ileride (7. bölümde) ayrıntılı konuşulacak kilitlenme şartlarıyla bağladı:

- Tut ve bekle (hold and wait) vardır. Herkes bir çubuğu tutarken diğerini bekler.
- Döngüsel bekleme (circular wait) vardır.
- Kesinti yoktur (no preemption). Bir sonraki kaynağı alamadığınız için elinizdekini bırakmıyorsunuz.

Böylece kilitlenmenin dört şartı da sağlanmış olur. Herkes birbirini bekler ve kimsenin bir adım atma şansı kalmaz.

> **Hocanın vurgusu:** Kilitlenme her zaman oluşmaz. Ancak sistemdeki herkes aynı anda birinci adımı yapıp ikinciye geçemediğinde oluşur. Kötülüğü de buradadır: Olma ihtimali vardır, ama her zaman olmak zorunda değildir. Eş zamanlı (concurrent) programlamanın asıl zorluğu da böyle hataları ayıklamanın (debugging) zor olmasıdır. Hatayı doğrudan göremeyebilirsiniz.

4.4 Kilitlenmeden Kaçınma Yolları

Hoca birkaç alternatif çözüm saydı:

- Masaya hiçbir zaman beş filozof oturtmamak, yani en fazla dört filozofa izin vermek.
- Bir filozofun çubukları yalnız iki taraftaki çubuğu da alabilecek durumdaysa almasına izin vermek. Bunun için çubuk alma işi bir kritik bölgeye konur ve o sırada diğerlerinin denemesi engellenir.
- Bir tür önceliklendirmeye gitmek, yani asimetrik bir çözüm kullanmak.

Hoca asimetrik çözümü şöyle anlattı: Tek numaralı filozoflardan başlanır. Tek numaralı filozoflar önce soldaki, sonra sağdaki çubuğu alır. Çift numaralılar daha sonra dener. Hocaya göre bu durumda bir dengesizlik oluşur. İlk turda 1 ve 3 çubukları dener, öbür turda 0, 2 ve 4. Özellikle 0, 2 ve 4 arasında yiyemeyen, yani açlığa düşen bir filozof olabilir. Hoca bu yöntemin daha da detaylandırılması gerektiğini söyledi.

> **Ders sonrası not:** Hoca asimetrik çözümü “çift numaralılar daha sonra dener” diye anlattı. Kitaptaki çözümde büyük olasılıkla çift numaralılar sırayla beklemez. Çubukları ters sırada, önce sağ sonra sol olarak alırlar. Bu sıralama döngüsel beklemeyi kırar. Hocanın “çift numaralılar aç kalabilir” kaygısı kitaptaki bu çözümün kendisinden doğmaz.

5. Semafor Kullanım Hataları

Hoca, monitörlere geçmeden önce semaforlarla yapılan tipik hataları hatırlattı. Bir semafor üzerinde normalde önce `wait`, sonra `signal` yapılır. Semafor da buna uygun ilklendirilmiş olmalıdır. Aksi durumlar sıkıntılıdır:

- Semafor düzgün ilklendirildiği hâlde önce `signal` sonra `wait` demek anlamlı değildir.
- `wait` ve `signal` yerine `wait wait` ya da `signal signal` demek açık programlama hatalarıdır.
- Bunlardan birinin kullanılmasını unutmak da hatadır. Yalnız yanlış yerde kullanmak değil, unutmak da hatadır.

Bu hataların hepsi kilitlenme ya da açlık olarak geri döner. Bu sorunları en aza indirmek ve daha kullanıcı dostu bir yapı sunmak için monitör yapısı önerilmiştir.

Hoca "bu aşk hikâyesinin sonunu" baştan söyledi: Monitörde de benzer durum geçerlidir. Monitörle semaforun bir üst seviye özetini, yani soyutlamasını oluşturursunuz. Bu soyutlama semaforun iç işleyişini gizler ve daha rahat kullanım sağlar. Ama monitör yanlış kullanılırsa ya da kullanılması unutulursa yine aynı sıkıntılar çıkar. Uygulama geliştirici senkronizasyonu doğru kullanmadığı sürece kilitlenme ve açlıkla karşılaşılabilir.

6. Monitörler

6.1 Yapı

Hocaya göre monitöre bakınca bir sınıf (class) gibi bir yapı görülür. Monitörün içinde paylaşılan değişken tanımları vardır. Bu değişkenler üzerindeki bütün işlemler monitörde tanımlanan prosedürlerle (procedure) gerçekleştirilir. Değişkenleri ilklendirmek için de sınıftaki yapıcıya (constructor) benzeyen bir ilklendirme kodu (initialization code) vardır. Semafor kullanırken ilk değer verdiğimiz gibi, monitördeki değişkenlere de bu kısımda ilk değer verilir.

Monitör teknik olarak şunu sağlar: Herhangi bir t anında monitörün içinde yalnız bir süreç aktif olabilir.

6.2 Koşul Değişkenleri

Hoca, monitörlerin bazı senkronizasyon problemlerinin çözümü için tek başına çok etkili olmayabileceğini söyledi. Daha "terzi işi" çözümler gerektiğinde koşul değişkenleri (condition variable) kullanılır. Bunlar genelde `condition x, y;` gibi tanımlanır. Üzerlerinde iki işlem yapılır:

- `x.wait()`: Semafordaki `wait`'e çok benzer. Çağıran süreç, aynı koşul üzerinde başka bir süreç `signal` yapana kadar bekletilir.
- `x.signal()`: Bu koşul üzerinde `wait` ile beklemeye geçmiş süreçlerden birinin yeniden çalışmasını sağlar.

Semafordan bir farkı vardır: O koşul üzerinde bekleyen hiçbir süreç yoksa `signal` çağrısının hiçbir etkisi yoktur.

> **Ek bilgi (derste anlatılmadı):** Semaforda `signal` her zaman semaforun değerini artırır. Bu yüzden semafora önceden yapılan bir `signal`, sonraki bir `wait`'in beklemeden geçmesini sağlar. Koşul değişkeninde ise bekleyen yokken yapılan `signal` kaybolur ve "hatırlanmaz". Bu yüzden koşul değişkenleri genelde monitördeki bir durum değişkeniyle birlikte kullanılır.

Koşul değişkenleriyle monitörün bir giriş kuyruğu olur. Bunun yanında her koşul için de ayrı bir bekleme kuyruğu oluşur. Böylece yapı daha özel ihtiyaçlara göre şekillendirilebilir.

6.3 Signal and Wait, Signal and Continue

Monitörde aynı anda tek süreç aktif olmalıdır. Bir P süreci `x.signal()` yaptığında, `x.wait()` ile askıya alınmış bir Q süreci varsa ne olur? İki sürecin monitörde aynı anda aktif olmasını istemiyoruz. Q çalışmaya devam edecekse P bekletilmelidir. Bunun için iki temel yaklaşım vardır:

- **Signal and wait:** `signal`'ı çağıran P beklemeye başlar, Q serbest bırakılır ve çalışmaya devam eder. P, Q monitörden çıkana kadar bekler.
- **Signal and continue:** P `signal`'ı çağırır ama çalışıp işini bitirir. P monitörden tamamen çıktıktan sonra Q çalışmaya devam eder.

Hoca, farklı programlama dillerinde farklı yaklaşımların seçildiğini, örneğin C# gibi dillerde farklı uyarlamalar görülebileceğini söyledi. Bu yüzden kullanılan yapıda uyarlamanın nasıl olduğuna bakmak gerekir.

7. Monitörle Filozoflar Çözümü

Hoca filozoflar problemini monitörle, kilitlenmesiz (deadlock-free) bir versiyonla çözdü.

Filozoflar ya düşünür ya yer. Ama çözümde üç durum (state) tanımlanmıştır: düşünme (`THINKING`), açlık (`HUNGRY`) ve yeme (`EATING`). Acıktığını anlama ile yemeye başlama arasındaki adım ayrı bir durum olarak bölünmüştür. Durumun tipi bir sayım (enumeration) olarak tanımlıdır. Her filozof için ayrıca `self` adında, koşul tipinde bir dizi vardır.

Monitördeki prosedürler şunlardır:

- `pickup(i)`: Filozof önce aç olduğuna karar verir ve kendi durumunu `HUNGRY` yapar. Sonra kendi için `test` işlemi yapar. Yemeye başlayamıyorsa kendi koşul değişkeninde bekler.
- `test(i)`: Kendinden önceki ve sonraki filozofun yemediğini kontrol eder. Kendisi de açsa durumunu `EATING` yapar ve devam eder.
- `putdown(i)`: Yemeyi bitiren filozofun çubukları bırakmasını sağlar.
- İlklendirme kısmında bütün filozoflar `THINKING` durumundadır.

```
monitor DiningPhilosophers {
    enum { THINKING, HUNGRY, EATING } state[5];
    condition self[5];

    void pickup(int i) {
        state[i] = HUNGRY;
        test(i);
        if (state[i] != EATING)
            self[i].wait();
    }

    void putdown(int i) {
        state[i] = THINKING;
        test((i + 4) % 5);
        test((i + 1) % 5);
    }

    void test(int i) {
        if (state[(i + 4) % 5] != EATING &&
            state[i] == HUNGRY &&
            state[(i + 1) % 5] != EATING) {
            state[i] = EATING;
            self[i].signal();
        }
    }

    initialization_code() {
        for (int i = 0; i < 5; i++)
            state[i] = THINKING;
    }
}
```

Her filozof i, monitörü kullanırken önce `pickup(i)` der. Bu çağrı çubukları almayı dener. Sonra filozof yer ve ardından `putdown(i)` der.

Hoca bu yapıyla kilitlenmenin çözüldüğünü, ama hâlâ bir problem olduğunu söyledi: açlık. Açlığın nereden kaynaklandığını ve nasıl çözülebileceğini sınıfa sordu. Cevap gelmeyince bunun bir alıştırma olmasını istedi. Öğrencilerin bu soru üzerine düşünüp bir çözüm üretmesini istedi (bkz. Örnek Sorular).

Hoca ayrıca monitörün çoğu zaman bir kütüphane olarak sunulduğunu söyledi. Monitörün üst

katmanda sunulabilmesi için altta başka bir senkronizasyon mekanizması kullanılmalıdır. Bu da genelde semafordur. Semaforlar konuşulduğu için hoca monitörün semaforla uyarlanmasının ayrıntısına girmedi.

8. Monitörde Bekleyen Süreçlerin Sırası ve Kullanım Hataları

8.1 Hangi Süreç Devam Edecek?

Önceki P ve Q örneği bire bir bir durumdu. Oysa aynı koşul değişkeni üzerinde birden fazla süreç bekleyebilir. Bir süreç `signal` dediğinde bunlardan hangisinin devam edeceğine karar vermek gerekir. Hoca iki yaklaşım saydı:

- **İlk gelen ilk hizmet alır (first come first served, FCFS):** Bekleyenler bir kuyrukta tutulur. Koşul üzerinde ilk `wait` yapan ilk devreye alınır.
- **Öncelikli bekleme:** Süreç `wait` işlemini yaparken bir öncelik değeri verir. Hocanın örneğinde bu değer, sürecin kaynağa ne kadar süreyle sahip olmak istediğidir. Biri 10, biri 5, biri 25 milisaniye dediyse en kısa olana öncelik verilebilir.

Slaytta bu yapı, bir kaynağı ayıran monitör örneği üzerinden gösterilmişti.

8.2 Monitörde de Aynı Hatalar Yapılabilir

Hoca, bütün bu yapıya rağmen semafordaki kullanım sıkıntılarının hâlâ geçerli olduğunu vurguladı:

- Kritik bölgeye monitör üzerinden girilmesi istenir, ama programcılardan biri monitörü kullanmayı unutur ve doğrudan kritik bölgeye dalar.
- Kaynağı koşul değişkeni üzerinden alan süreç kaynağı hiç bırakmayabilir. Bu, semaforda `signal`'ı unutmaya karşılık gelir.
- Süreç, hiç almadığı bir kaynağı bırakmaya kalkabilir. Bu da `wait` demeden `signal` demektir.
- Semaforda iki kez `wait` demek gibi, monitörde aynı koşul değişkeni üzerinde iki kez `wait` denebilir.

  Hocanın vurgusu: Monitör bir soyutlama sağlar ve bazı şeyleri kolaylaştırır, ama sizi kullanım hatalarından arındırmaz. Bunu aklınızın bir köşesinde tutun.

9. İşletim Sistemlerinde Senkronizasyon Örnekleri

Hoca bu kısma hızlıca baktı. Koşul değişkenleriyle ilgili Pthreads kütüphanesi örnekleri uygulama saatlerinde gösterilecekti.

9.1 Solaris

Solaris'te ilginç bir kavram vardır: uyarlanabilir mutex (adaptive mutex). Önceki derslerde semaforların iki şekilde uyarlanabileceği konuşulmuştu. Biri dönen kilit (spinlock) olarak, diğeri bekleyecek süreçleri bekleme durumuna (waiting state) aktararak. Solaris bu noktada uyarlanabilir bir yaklaşım izler. Mutex'i dönen kilitle sağlar. Ama beklemenin uzun süreceği düşünüldüğünde bunu diğer yapıya, yani süreci uyutan yapıya çevirir. Solaris bunun dışında koşul değişkenleri ve okuyucu-yazıcı kilitleri de sunar.

> **Ek bilgi (derste anlatılmadı):** Kitaba göre Solaris'teki karar, kilidi tutan iş parçacığının durumuna bakılarak verilir. Kilidi tutan iş parçacığı başka bir işlemcide çalışıyorsa kilidin kısa sürede bırakılması beklenir ve bekleyen iş parçacığı döner. Kilidi tutan çalışmıyorsa bekleyen iş parçacığı uyutulur. Bu yöntem çok işlemcili sistemlerde anlamlıdır.

Soru: Solaris hâlâ yaygın kullanılıyor mu? **Cevap:** Hoca bunun "yaygın"dan ne kastedildiğine bağlı olduğunu söyledi. Sunucu sistemlerinde AIX, Linux ve HP-UX gibi sistemler hâlâ bir yerlerde kullanılıyor. Masaüstünde Linux gibi kullanılmasını düşünüyorsanız böyle bir şey pek yok. Bu Unix türevlerinin üreticilerinde de Linux'a doğru bir geçiş var. IBM'in birçok ürünü doğrudan Linux desteği sağlıyor. Bazı sistemler Linux çalıştırılabilir dosyalarını sınırlı da olsa doğrudan çalıştırmaya imkân tanıyan ikili uyumluluk (binary compatibility) sunuyor. Hoca bu

konuda HP-UX'ten emin olmadığını belirtti. Sonuç olarak bu sistemler ortadan kalkmış değil, ama daha niş ürünlerde kullanılıyor.

9.2 Windows

Hoca, Windows'un çok işlemcili sistemlerde doğrudan dönen kilit kullandığını söyledi.

> **Ders sonrası not:** Kitaba göre Windows çekirdeği büyük olasılıkla tek işlemcili sistemde global kaynaklara erişirken kesmeleri maskeler, çok işlemcili sistemde dönen kilit kullanır. Kullanıcı düzeyinde ise dağıtıcı nesneleri (dispatcher objects) sunar: mutex, semafor, olay (event) ve zamanlayıcı (timer).

9.3 Linux

Hoca, Linux'ta 2.6 sürümünden sonraki duruma baktı ve 2.6'nın çıkışının üzerinden çok zaman geçtiğini hatırlattı. Linux semafor ve atomik tam sayılar (atomic integer) sunar. Atomik tam sayılarla tam sayılar üzerindeki işlemler atomik olarak yapılabilir. Semafor uyarlaması dönen kilit üzerinden sağlanır. Yalnız tek işlemcili yapılarda dönen kilit yerine çekirdek kesintisi (kernel preemption) kapatılıp açılır. Hocaya göre 2.6'dan sonra çekirdeğin tamamı kesintili (preemptive) hâle gelmiştir. Linux okuyucu-yazıcı kilitlerini de sağlar.

9.4 Pthreads

Pthreads yapısında mutex kilitleri, koşul değişkenleri ve okuyucu-yazıcı kilitleri vardır. Hoca bunların uygulama saatinde gösterileceğini söyledi ve ayrıntıya girmedi.

10. CPU Zamanlamaya Giriş

Aradan sonra hoca CPU zamanlamaya (CPU scheduling) geçti. Bölümde önce zamanlamanın kavramsal tanımı yapılacak, sonra zamanlama ölçütleri, algoritmalar ve algoritmaların bu ölçütlere göre başarımı konuşulacaktı.

Hoca bölümün planını şöyle anlattı:

- Süreç (process) ve iş parçacığı (thread) zamanlaması ayrı ayrı konuşulacaktır. Bu yüzden ayrıca bir iş parçacığı zamanlaması başlığı vardır.
- Baştaki ölçütler ve algoritmalar öncelikle tek işlemcili yapılar için ortaya atılmıştır. İşlemci sayısı arttıkça farklı yaklaşımlar ortaya çıkmıştır. Bu yüzden önce tek işlemci, sonra çok işlemci durumu konuşulacaktır.
- Gerçek zamanlı CPU zamanlaması (real-time CPU scheduling) kabaca geçilecektir.
- Son olarak güncel işletim sistemlerinin zamanlama açısından sundukları konuşulacaktır.

10.1 CPU ve G/Ç Patlamaları

Hoca çoklu programlamanın (multiprogramming) amacını hatırlattı: İşlemci zamanını daha etkili kullanmak. Bir sürecin CPU'ya ihtiyacı olmadığı anda orada başka bir sürecin çalıştırılması sağlanır.

Her süreç bir diyagramla ifade edilebilir. Süreç başladığında bir CPU patlamasından (CPU burst) geçer. CPU patlaması, CPU'da çalıştırılacak işlemlerin olduğu kısımdır. G/Ç patlaması (I/O burst) ise G/Ç işleminin gerçekleşmesi için geçen süredir. Bir süreç birçok CPU ve G/Ç patlamasından oluşur.

- CPU patlamalarının toplamı daha fazlaysa sürece CPU'ya bağlı (CPU-bound) denir.
- G/Ç patlamaları daha fazlaysa sürece G/Ç'ye bağlı (I/O-bound) denir.

Hoca, zamanlayıcılar konuşulurken bunun kısaca geçildiğini hatırlattı. Uzun vadeli zamanlayıcı (long-term scheduler) uygun seçim yapmazsa, örneğin hep G/Ç'ye bağlı süreçleri seçerse, süreçler kuyruktan kuyruğa bir tren gibi birbirini izleyebilir.

Bir CPU patlamasından sonra bir G/Ç patlaması gelir. Süreç G/Ç için beklemelidir. Çevre birim G/Ç işlemini sürecin yerine yapar, tamamlayınca haber verir. Bu sırada CPU boşta (idle)

kalır. Çoklu programlamada amaç, bu sırada başka bir sürecin CPU patlamasını işlemcide çalıştırarak CPU zamanını değerlendirmektir.

10.2 CPU Patlama Sürelerinin Dağılımı

Hoca slayttaki histogramı açıkladı. Bu grafik görecelidir: Patlama süreleri işlemci hızına ve mimariye göre değişir. Daha yavaş bir işlemcide grafik biraz sağa kayar. Ama genel karakteristik her sistemde benzerdir.

Yatay eksende milisaniye cinsinden CPU patlama süresi vardır: 0, 8, 16, 24, 32, 40. Dikey eksende frekans vardır: 20, 40, …, 160'a kadar gider, ama 160'a ulaşan bir değer yoktur. Frekansı en yüksek olanlar 0 ile 8 ms arasındaki patlamalardır. Hocaya göre ortalarsak 4 ms'nin altındaki patlamalar daha fazladır. Çok uzun CPU patlamaları sistemde pek görülmez.

Bu dağılım, CPU zamanlama algoritmalarını değerlendirirken sistemin hangi yükle test edilmesi gerektiğini görmek için önemlidir.

11. CPU Zamanlayıcı, Kesintili ve Kesintisiz Zamanlama

11.1 Kısa Vadeli Zamanlayıcı

Daha önce üç zamanlayıcı konuşulmuştu: kısa vadeli (short-term), uzun vadeli (long-term) ve orta vadeli (medium-term). CPU zamanlayıcı denince doğrudan kısa vadeli zamanlayıcı kastedilir. Kısa vadeli zamanlayıcı, hazır kuyruğunda (ready queue) bekleyen süreçlerden birini işlemciyi kullanmak üzere seçer. İşlemcinin kullanımına doğrudan karar verdiği için CPU zamanlayıcı diye anılır.

Hoca diğer ikisini de hatırlattı. Uzun vadeli zamanlayıcı ikincil bellekteki (secondary storage) işlerden hangilerinin belleğe geleceğini belirler. Orta vadeli zamanlayıcı ise oluşabilecek dengesizlikleri önlemek için kısmen çalışmış süreçleri bellekten dışarı atabilir (swap out) ya da geri getirebilir (swap in).

11.2 Zamanlama Kararının Verildiği Dört Durum

İşlemcinin sonraki adımda hangi süreç tarafından kullanılacağına dört durumda karar verilir. Hoca bunları tersten sayarak anlattı:

1. Çalışan (running) bir süreç bekleme (waiting) durumuna geçer. Örneğin bir sistem çağrısı yapar ya da bir semafor üzerinde `wait` işlemi yapar.
2. Çalışan bir süreç hazır (ready) durumuna geçer, yani işlemciyi bırakır.
3. Bekleyen bir süreç hazır durumuna geçer. Örneğin bir G/Ç işlemi başlatmıştır. G/Ç bitene kadar bekleme durumundadır. G/Ç bittiğine dair haber gelince hazır durumuna geçer.
4. Çalışan süreç kendi isteğiyle ya da başkasının isteğiyle sonlanır (terminate).

Bu dört durumdan yalnız 1 ve 4 kullanılıyorsa zamanlama kesintisizdir (non-preemptive). İşlemciye atanan süreç ya sonlanana ya da örneğin bir G/Ç başlatarak kendi isteğiyle bekleme durumuna geçene kadar çalışır. Siz onu hiçbir şekilde kesmezsiniz. Diğer durumlar da işin içine katılıyorsa zamanlama kesintilidir (preemptive). Yani bir sürecin çalışmasını kesip işlemciyi başka bir sürece verebilirsiniz.

> **Ders sonrası not:** Hoca durumları tersten saydığı için hangi numaranın hangi duruma ait olduğu anlatımda karışık. Yukarıdaki numaralama kitaptaki numaralamadır. Hocanın "yalnız 1 ve 4" sözü bu numaralamaya göre doğrudur: Kesintisiz zamanlama yalnız çalışan → bekleyen geçişinde ve sonlanmada karar verir.

11.3 Kesintili Zamanlamanın Getirdiği Sorunlar

Kesintili zamanlama yapılırken düşünülmesi gereken durumlar vardır:

- Paylaşılan veriye (shared data) erişilirken kesinti olursa bir senkronizasyon mekanizması kurmak gerekir. Senkronizasyon bu yüzden bol bol konuşuldu.

- Çekirdek kipinde (kernel mode) kesintiye izin verilip verilmeyeceği düşünülmelidir. Çekirdekteki ortak alanlara da erişildiği için çekirdek kipinde kesinti ayrı bir sıkıntıdır. Kesinti yaparsanız birinci maddedeki durumun aynısını çekirdek kipinde yaşarsınız.
- İşletim sisteminin çalışması sırasında oluşan kesmelerin (interrupt) ele alınması da üzerine düşünülmesi gereken bir konudur.

12. Dağıtıcı ve Dağıtım Gecikmesi

Zamanlayıcı, sonraki adımda çalışacak süreci seçer. Kısa vadeli zamanlayıcının seçtiği sürece işlemcinin kontrolünü vermek için gereken işleri yapan modüle dağıtıcı (dispatcher) denir. Bu işler şunlardır:

- Bağlam değişimi (context switch) yapmak. Çalışmakta olan sürecin son durumu alınır ve süreç kontrol bloğuna (process control block, PCB) yazılır. Sonra çalıştırılmak için seçilen sürecin kontrol bloğundaki bilgiler alınır ve yazmaç kümesi (register set), program sayacı gibi gerekli yerlere yazılır.
- Kullanıcı kipine (user mode) geçmek.
- Programın kaldığı yeri işaretleyip çalışmasını oradan devam ettirmek.

Bu işlerin hepsi için geçen toplam süreye dağıtım gecikmesi (dispatch latency) denir. İş dağıtıcı tarafından yapıldığı için bu ad verilmiştir.

13. Zamanlama Ölçütleri

Hoca, zamanlama için birçok ölçüt (criteria) olduğunu, ama hepsinin bir anda kullanılmadığını söyledi. Her biri farklı sistem tipleri için önemlidir.

- **CPU kullanımı (CPU utilization):** CPU zamanının ne kadarının kullanıldığını gösterir. Çoğu sistemde kullanılabilir. Hoca özellikle sunucu sistemlerini örnek verdi: Sunucu ve CPU için ciddi bir para ödenmiştir ve CPU zamanının boşa geçmesi istenmez. Günümüzde bulut sistemlerde sunucu kiralanır ve çoğu zaman kiralanan şey sunucunun CPU zamanıdır.
- **Verim (throughput):** Birim zamanda tamamlanan süreç sayısıdır.
- **Tamamlanma süresi (turnaround time):** Bir süreç açısından, sürecin sisteme verilmesinden (submit) tamamlanıp sistemden çıkmasına kadar geçen süredir.
- **Bekleme süresi (waiting time):** Zamanlayıcının bir sürecin CPU'da çalışma süresi üzerinde etkisi yoktur. Süreci aynı CPU'da daha hızlı ya da daha yavaş çalıştıramaz. Ama fazladan bağlam değişimleriyle ve süreci bekleterek tamamlanma süresini uzatabilir. Zamanlayıcının etkisi, sürecin hazır kuyruğunda geçirdiği süre üzerindedir. Bekleme süresi, tamamlanma süresinden sürecin çalışma süresinin çıkarılmış hâli gibi düşünülebilir. Yani hazır kuyruğunda geçen süredir.
- **Yanıt süresi (response time):** Tamamlanma süresi üzerinde hem bekleme hem çalışma süresinin etkisi vardır. Oysa etkileşimli (interactive) bir sistemde önemli olan, sistemin bizim etkimize tepki verip vermediği ve bu tepkiyi ne kadar sürede verdiğidir. Yanıt süresi, sürecin sisteme verilmesinden ilk cevabın oluşmasına kadar geçen süredir.

Amaçlar şöyledir:

Ölçüt	Amaç
CPU kullanımı	en büyük
Verim	en büyük
Tamamlanma süresi	en küçük
Bekleme süresi	en küçük
Yanıt süresi	en küçük

Algoritmalar bu ölçütlere göre değerlendirilir. Örneklerde kullanılan bağıntılar şunlardır:

$$\text{tamamlanma süresi} = \text{bitiş zamanı} - \text{geliş zamanı}$$

$$\text{bekleme süresi} = \text{tamamlanma süresi} - \text{CPU patlama süresi}$$

14. İlk Gelen İlk Hizmet Alır (FCFS)

İlk algoritma, yapı olarak oldukça basit olan ilk gelen ilk hizmet alır (first come first served, FCFS) algoritmasıdır.

> **Hocanın vurgusu:** Örneklerde süreçlerin patlama süreleri (burst time) verilir. Ama bu süreler önceden bilinmez. Süreç çalışmaya başlamadan patlamasının ne olacağını doğrudan bilemezsiniz. Bunlar tahmini değerlerdir.

14.1 Örnek: 24, 3, 3

Süreçler P_1, P_2, P_3 sırasıyla geliyor. Patlama süreleri 24, 3, 3'tür.

Gantt diyagramı:

Süreç	P_1	P_2	P_3
Aralık	0-24	24-27	27-30

P_1 ilk geldiği için $t = 0$'da başlar ve 24 birim çalışır. Sonra P_2 3 birim, ardından P_3 3 birim çalışır.

- P_1 hiç beklemez: bekleme 0.
- P_2, P_1'in bitmesini bekler: bekleme 24.
- P_3 hem P_1'i hem P_2'yi bekler: bekleme 27.

$$\bar{W} = \frac{0 + 24 + 27}{3} = \frac{51}{3} = 17$$

Hocaya göre bu ortalama "bayağı yüksek"tir. P_1 uzun bir patlamaya sahiptir. P_2 ve P_3 ise kısa işlerdir. Hocanın deyişiyle bunlar "bir bakıp çıkacak" türden işlerdir, P_1 ise içeride bayağı vakit geçirecektir. Uzun iş önce geldiği için hemen başlatılmış, kısa işler uzun süre bekletilmiştir.

14.2 Aynı Süreçler, Farklı Sıra

Hoca, süreçler farklı sırada gelseydi ne olacağını sordu. Sıra P_2, P_3, P_1 olsun:

Süreç	P_2	P_3	P_1
Aralık	0-3	3-6	6-30

- P_2 doğrudan çalışır: bekleme 0.
- P_3, P_2'nin bitmesini bekler: bekleme 3.
- P_1 en son çalışır, P_2 ve P_3'ü bekler: bekleme 6.

$$\bar{W} = \frac{6 + 0 + 3}{3} = \frac{9}{3} = 3$$

Kimse fazladan beklememiştir.

> **Ders sonrası not:** Hoca önce "P3, P2, P1 diye gelmiş olsalardı" dedi, ama ardından P_2'yi önce, P_3'ü sonra çalıştırdı. Verdiği bekleme değerleri (0, 3, 6) P_2, P_3, P_1 sırasına aittir. Sıra P_3, P_2, P_1 olsaydı da ortalama yine 3 olurdu, yalnız P_2 ile P_3'ün bekleme değerleri yer değiştirirdi.

14.3 Konvoy Etkisi

FCFS'nin sorunu, gelen sıraya göre çalışırken araya uzun süreçlerin girmesidir. Hoca bir P_4 daha olduğunu ve onun da kısa olduğunu düşünmemizi istedi. Uzun bir işin arkasına düşerse onun için de kaçınılmaz olarak aşırı bir bekleme süresi oluşur. Sistemin ortalama bekleme süresi "fezaya varır".

Hoca bunu patlamalar üzerinden de açıkladı. P_2 ve P_3 kısa bir CPU patlaması yapıp sonra G/Ç başlatacak olsun. P_1'in çok uzun CPU patlaması, bunların CPU patlamalarını tamamlayıp G/Ç başlatmasını engeller. Böylece eş zamanlılık (concurrency) kaybedilir. Kısa işler önde olsaydı CPU'daki işlerini hızla bitirir, G/Ç'ye geçerdi. Onlar G/Ç ile uğraşırken P_1 CPU'da çalışırdı. Uzun patlama araya girince ise önce P_1 CPU'da çalışır, sonra büyük olasılıkla G/Ç'ye gider. P_2 ve P_3 çalışır, sonra G/Ç'de P_1'in arkasında beklemeye girer. Yani kısa süreçler uzun sürecin arkasında bir konvoy oluşturur. Buna konvoy etkisi (convoy effect) denir.

Soru: Bir öğrenci burada konunun yanıt süresiyle (responsiveness) ilgisini sordu. **Cevap:** Hoca şu an konuşulanın yanıt süresiyle ilgisi olmadığını, hâlâ bekleme süresinin konuşulduğunu söyledi.

15. En Kısa İş Önce (SJF) ve Sonraki Patlamanın Tahmini

Araya giren uzun işler bekleme ve konvoy açısından sorun çıkarıyorsa işler önceliklendirilebilir. Önceliklendirmek istediğimiz şey kısa işlerdir. Yani CPU patlaması kısa olan süreçler önce çalıştırılır. Bu, en kısa iş önce (shortest job first, SJF) algoritmasıdır.

15.1 Sonraki CPU Patlamasının Tahmini

Hoca önce ileride kalan bir slayta atladı. Çalıştırmadığınız bir sürecin CPU patlamasının ne olacağını bilemezsiniz. Bu yüzden onu tahmin etmeniz gerekir. Bunun için basit bir süzgeç (filter) kullanılabilir. Süzgecin işi, önceki değerlerden bir sonraki patlamanın ne olacağını kestirmektir.

Grafikte mavi eğri tahmini (guess) τ_i, diğeri gerçekleşen patlamayı t_i gösterir. Sonraki tahmin, bir önceki tahmin ile bir önceki gerçekleşen değer arasında α ve $1-\alpha$ ağırlıklarıyla kurulur:

$$\tau_{n+1} = \alpha\, t_n + (1-\alpha)\, \tau_n, \qquad 0 \le \alpha \le 1$$

$\alpha = 1/2$ alınırsa iki değere eşit ağırlık verilir. Hocanın örneği, $\tau_0 = 10$ ve gerçekleşen patlamalar 6, 4, 6, 4, 13, 13, 13 içindir. Geçmişe dair hiçbir bilgi olmadığı için ilk tahmin 10'dur.

- $\tau_1 = \frac{6}{2} + \frac{10}{2} = 3 + 5 = 8$
- $\tau_2 = \frac{4}{2} + \frac{8}{2} = 2 + 4 = 6$
- $\tau_3 = \frac{6}{2} + \frac{6}{2} = 6$. Hocanın deyişiyle burada tahmin "denk düştü".
- $\tau_4 = \frac{4}{2} + \frac{6}{2} = 5$
- $\tau_5 = \frac{13}{2} + \frac{5}{2} = 9$. Gerçek patlama bir anda 13'e çıkmıştır.
- $\tau_6 = \frac{13}{2} + \frac{9}{2} = 11$
- $\tau_7 = \frac{13}{2} + \frac{11}{2} = 12$ (ara adım eklendi)

n	0	1	2	3	4	5	6	7
t_n	6	4	6	4	13	13	13	
τ_n	10	8	6	6	5	9	11	12

Tahmin gerçek değerlere yavaş yavaş yakınsar. Hocanın deyişiyle hem düşüşe hem yükselişe alışırsınız. Değerler python ile doğrulandı.

> **Ders sonrası not:** Hoca ilk adımda "10 bölü 2 5, 6 bölü 3" dedi. Doğrusu $6/2 = 3$'tür. Hoca sonucu doğru olarak 8 buldu.

Ek bilgi (derste anlatılmadı): α'nın uç değerleri formülün anlamını gösterir. $\alpha = 0$ ise $\tau_{n+1} = \tau_n$ olur ve yakın geçmiş hiç hesaba katılmaz. $\alpha = 1$ ise $\tau_{n+1} = t_n$ olur ve yalnız son gerçek patlama dikkate alınır. Formül açılırsa eski patlamaların ağırlığı $(1-\alpha)$'nın kuvvetleriyle üstel olarak azalır. Bu yüzden yönteme üstel ortalama (exponential average) denir.

15.2 SJF Örneği

Patlama süreleri bu şekilde tahmin edilmiş süreçler olsun. $t = 0$ anında P_1, P_2, P_3, P_4 sistemdedir. Patlamalar: $P_1 = 6$, $P_2 = 8$, $P_3 = 7$, $P_4 = 3$.

En kısa patlama P_4'ündür. Sıra P_4, P_1, P_3, P_2 olur:

Süreç	P_4	P_1	P_3	P_2
Aralık	0-3	3-9	9-16	16-24

- P_4 doğrudan çalışır: bekleme 0.
- P_1 ikinci sırada çalışır, ilk çalışanın patlaması 3'tür: bekleme 3.
- P_3, P_4 ve P_1'den sonra çalışır: bekleme $3 + 6 = 9$.
- P_2 en uzun iş olduğu için herkes bittikten sonra çalışır: bekleme $3 + 6 + 7 = 16$.

$$\bar{W} = \frac{3 + 16 + 9 + 0}{4} = \frac{28}{4} = 7$$

Ders sonrası not: Hoca P_1'in beklemesini açıklarken "ilk sırada çalıştırılan P3" dedi. İlk çalışan süreç P_4'tür (patlaması 3).

15.3 Aynı Süreçlerin FCFS ile Çalışması

Hoca, aynı süreçler bu sırayla gelip FCFS ile çalışsaydı ne olacağını sordu:

Süreç	P_1	P_2	P_3	P_4
Aralık	0-6	6-14	14-21	21-24

- P_1: bekleme 0.
- P_2: P_1'in bitişini bekler, bekleme 6.
- P_3: P_1 ve P_2'yi bekler, bekleme $6 + 8 = 14$.
- P_4: bekleme $6 + 8 + 7 = 21$.

$$\bar{W} = \frac{0 + 6 + 14 + 21}{4} = \frac{41}{4} = 10{,}25$$

Hoca sonucu "on küsur" diye bıraktı. SJF ile ortalama bekleme 10,25'ten 7'ye inmiştir. Değerler python ile doğrulandı.

Ek bilgi (derste anlatılmadı): SJF, verilen bir süreç kümesi için en küçük ortalama bekleme süresini veren algoritmadır. Kısa bir işi uzun bir işin önüne almak, kısa işin beklemesini uzun işin beklemesinin arttığından daha fazla azaltır. Ama SJF'nin gerçek uygulanması bir sonraki patlamanın bilinmesine bağlıdır. Bu yüzden pratikte yukarıdaki gibi tahmin kullanılır.

15.4 SJF de Bir Öncelikli Zamanlamadır

Hoca, aslında bunların hepsinin öncelikli zamanlama (priority scheduling) olduğunu söyledi. Belli bir ölçüte göre çalışacak süreçler önceliklendirilir. SJF'de bu ölçüt patlama süresidir. Ama bununla sınırlı olmak zorunda değiliz.

16. En Kısa Kalan Süre Önce (SRTF)

Bir başka yaklaşım en kısa kalan süre önce (shortest remaining time first, SRTF) algoritmasıdır. Burada kalan patlama süresi en kısa olan süreç önce çalıştırılır.

Hoca buradaki dinamikliği vurguladı. SJF'de süreçler sisteme bir kez girer ve sistemde oldukları sürece önceliklendirme sabittir. Oysa her zaman adımında, süreçlerin çalışmasına bağlı olarak öncelikler değişebilir. SRTF, SJF'nin kesintili versiyonudur. Zamanlama kararının verilebileceği dört durumdan biri, hazır kuyruğuna yeni bir sürecin gelmesidir. SRTF'de her süreç geldiğinde yeniden karar verilir. Hoca bunu "filmi kare kare oynatmak" gibi her adımda yeniden karar vermeye benzetti.

16.1 Örnek

Süreçler (geliş zamanı, patlama süresi) olarak verilmiştir: P_1 (0, 8), P_2 (1, 4), P_3 (2, 9), P_4 (3, 5).

Hocanın adım adım anlatımı:

- $t = 0$: Sistemde yalnız P_1 vardır, onunla başlanır.
- $t = 1$: P_2 gelir. Patlaması 4'tür. P_1 bir birim çalıştığı için 7 birimi kalmıştır. P_2 daha kısadır, P_1 kesilir ve P_2 çalışır.
- $t = 2$: P_3 gelir. Patlaması 9 ile uzundur. P_2'nin kalanı 3'e inmiştir. P_2 devam eder.
- $t = 3$: P_4 gelir. Patlaması 5'tir. P_2'nin kalanı 2'dir. P_2 devam eder.
- $t = 5$: P_2 biter. Kalanlar $P_1 = 7$, $P_4 = 5$, $P_3 = 9$'dur. En kısa olan P_4 çalışır. Yeni süreç gelmediği için değişiklik olmaz.
- $t = 10$: P_4 biter. P_1'in kalan 7 birimi çalışır.
- $t = 17$: P_1 biter. En son P_3 çalışır ve $t = 26$'da biter.

Süreç	P_1	P_2	P_4	P_1	P_3
Aralık	0-1	1-5	5-10	10-17	17-26

Bekleme süreleri (ara adım eklendi):

Süreç	Geliş	Patlama	Bitiş	Tamamlanma	Bekleme
P_1	0	8	17	17	9
P_2	1	4	5	4	0
P_3	2	9	26	24	15
P_4	3	5	10	7	2

$$\bar{W} = \frac{9 + 0 + 15 + 2}{4} = \frac{26}{4} = 6{,}5$$

> **Ders sonrası not:** Hoca bu örnekte ortalama bekleme süresini söylemedi. Python ile yapılan hesapta Gantt sırası hocanın anlattığıyla aynı çıktı ve ortalama bekleme 6,5 birim (kitapta ms) bulundu.

17. Öncelikli Zamanlama ve Yaşlandırma

17.1 Öncelik Değeri

Hoca, SJF ve SRTF'nin de önceliklendirilmiş yapılar olduğunu hatırlattı. Önceliklendirme her şeye göre yapılabilir. Örneğin süreci başlatan kullanıcının önceliğine göre, patlama süresine göre ya da başka ölçütlere göre. Öncelik genelde bir tam sayı (integer) ile ifade edilir.

> **Hocanın vurgusu:** Öncelik değerinin yönü sisteme göre değişir. Bazı sistemlerde büyük tam sayı yüksek öncelik, bazılarında küçük tam sayı yüksek öncelik demektir. Slayttaki gibi olmak zorunda değildir, buna aldanmayın.

17.2 Açlık ve Bir Şehir Efsanesi

> **Hocanın vurgusu:** Öncelikli zamanlamada en önemli sorun, önceliği düşük olan bir sürecin açlığa (starvation) maruz kalmasıdır.

Hoca bununla ilgili, ne kadar doğru olduğunu bilmediği bir şehir efsanesi anlattı. MIT'de 1973'te bir IBM 7094 sistemi kapatılır. Sistemde 1967'de verilmiş düşük öncelikli bir süreç bulunur. Hoca ayrıca 1967 ile 1973 arasındaki altı yıl boyunca sistemin hiç kapanmamış olmasına dikkat çekti. Süreç orada öylece beklemiştir.

17.3 Yaşlandırma

En basit ve temel çözüm şudur: Bir süreç belli bir süre boyunca CPU'ya atanamıyorsa, yani bir türlü zamanlanamıyorsa, önceliği bir kademe yükseltilir. Hâlâ kaynak verilemezse bir adım sonra tekrar yükseltilir. Bu yönteme yaşlandırma (aging) denir. Burada ne kadar bekleneceği ve ne kadar yükseltileceği kritik kararlardır.

Hocanın örneği: Öncelik ölçeği 0 ile 127 arasında olsun. Küçük sayı yüksek öncelik ifade etsin, yani 0 en yüksek, 127 en düşük öncelik olsun. Sürecin önceliği her 15 dakikada bir artırılsın, yani değeri 127'den 126'ya, 125'e insin. 127'den 0'a çıkması için:

$$127 \times 15 = 1905 \text{ dakika} = 31{,}75 \text{ saat} \approx 32 \text{ saat}$$

Hocaya göre altı yıl beklemeye gerek yoktur, ama bu süre bile oldukça yüksektir. Bu karar, öncelikli bir sistem kullanılıyorsa sistemin kullanılma şekline bağlıdır.

17.4 Örnek

Beş süreç vardır. Patlama süreleri P_1'den P_5'e 10, 1, 2, 1, 5'tir. Bu örnekte en düşük değer en yüksek önceliktir: 1 en yüksek, 5 en düşük öncelik.

Çalışma sırası P_2, P_5, P_1, P_3, P_4'tür:

Süreç	P_2	P_5	P_1	P_3	P_4
Aralık	0-1	1-6	6-16	16-18	18-19

Bekleme süreleri: $P_1 = 6$, $P_2 = 0$, $P_3 = 16$, $P_4 = 18$, $P_5 = 1$. Hoca toplamı adım adım topladı: $16 + 18 = 34$, $34 + 6 = 40$, $40 + 1 = 41$.

$$\bar{W} = \frac{6 + 0 + 16 + 18 + 1}{5} = \frac{41}{5} = 8{,}2$$

Değerler python ile doğrulandı.

Örnek Sorular

Soru 1 (derste sözlü, 00:27:25): Filozoflar probleminin basit semafor çözümünde (her filozof önce `chopstick[i]`'yi, sonra `chopstick[(i+1) % 5]`'i bekler) bir problem var mı?

> **Çözüm (derste verildi):** Evet, kilitlenme olabilir. Beş filozof sırayla ilk `wait`'i yapıp hemen ardından bağlam değişimine uğrarsa herkes solundaki çubuğu tutar ve sağdakini bekler. Tut ve bekle, döngüsel bekleme ve kesinti olmaması şartları sağlanır. Kilitlenme her zaman oluşmaz, ama oluşma ihtimali vardır.

Soru 2 (derste sözlü, 00:45:35): Monitörle yazılan kilitlenmesiz filozoflar çözümünde açlık nereden kaynaklanır ve nasıl çözülebilir?

Çözüm (derste verilmedi): `test(i)` yalnız iki komşunun o anda yemiyor olmasına bakar. Bir filozofun ne kadar süredir aç olduğu hesaba katılmaz. Örneğin 1 numaralı filozof açken 0 ve 2 numaralı komşuları sırayla yemeye başlarsa, 1'in `test`'i her seferinde komşulardan biri yediği için başarısız olur. 0 bitirip tekrar acıktığında 2 hâlâ yiyorsa 0 yine yiyebilir, böylece 1 hiç yiyemeyebilir. Çözüm için açlığı süreye bağlamak gerekir. Örneğin aç bekleyen filozoflar için bir sıra (FIFO) tutulabilir ya da bir filozofun, kendisinden daha uzun süredir aç bekleyen bir komşusu varsa yemeye başlamasına izin verilmez. Bu, öncelikli zamanlamadaki yaşlandırma fikrine benzer.

Soru 3 (derste sözlü, 01:31:13): Patlamaları 24, 3, 3 olan P_1, P_2, P_3 bu sırayla gelip FCFS ile çalışırsa ortalama bekleme süresi nedir? Bu yapının sıkıntısı nedir?

Çözüm (derste verildi): Beklemeler 0, 24, 27, ortalama $51/3 = 17$. Sıkıntı, uzun işin önde olması yüzünden kısa işlerin uzun süre beklemesidir (konvoy etkisi).

Soru 4 (derste sözlü, 01:32:56): Aynı süreçler P_2, P_3, P_1 sırasıyla gelseydi ortalama bekleme süresi ne olurdu?

Çözüm (derste verildi): Beklemeler $P_1 = 6$, $P_2 = 0$, $P_3 = 3$, ortalama $9/3 = 3$.

Soru 5 (derste sözlü, 01:38:14): $\alpha = 1/2$, $\tau_0 = 10$ ve gerçekleşen patlamalar 6, 4, 6, 4, 13, 13, 13 iken sonraki patlama tahminleri nedir?

Çözüm (derste verildi): 8, 6, 6, 5, 9, 11. Hoca 11'de durdu. Son gerçek patlama 13 ile bir sonraki tahmin $13/2 + 11/2 = 12$ olur (python ile doğrulandı).

Soru 6 (derste sözlü, 01:40:30): Patlamaları $P_1 = 6$, $P_2 = 8$, $P_3 = 7$, $P_4 = 3$ olan süreçler SJF yerine FCFS ile çalışsaydı ortalama bekleme süresi ne olurdu?

Çözüm (derste verildi): Beklemeler 0, 6, 14, 21, ortalama $41/4 = 10{,}25$. Hoca "on küsur" dedi. SJF'de bu değer 7'dir.

Dersin Toparlaması

Dersin ilk yarısında senkronizasyon konusu kapandı. Sınırlı tampon problemi üç semaforla çözüldü: `mutex` tamponu korur, `empty` ve `full` boş ve dolu yerleri sayar. Okuyucu-yazıcı probleminin birinci varyasyonunda okuyucular önceliklidir. Çözüm kilitlenmeyi önler, ama yazıcıların açlığını önlemez. Filozoflar probleminin basit semafor çözümü kilitlenmeye yol açabilir. Masadaki filozof sayısını sınırlamak, iki çubuğu birlikte almak ya da asimetrik sıralama bu kilitlenmeyi önler. Monitörler semaforun üstünde bir soyutlama sunar: İçeride aynı anda tek süreç aktiftir ve koşul değişkenleriyle daha özel bekleme kuyrukları kurulur. Monitörle filozoflar kilitlenmesiz çözülür, ama açlık kalır. Hocanın sık tekrarladığı ders şudur: Mekanizmayı işletim sistemi sağlar, doğru kullanmak geliştiricinin işidir. Monitör de kullanım hatalarından korumaz.

İkinci yarıda CPU zamanlamaya girildi. Süreçler CPU ve G/Ç patlamalarından oluşur ve CPU patlamalarının çoğu kısadır. Kısa vadeli zamanlayıcı dört durumda karar verir. Yalnız çalışan → bekleyen geçişinde ve sonlanmada karar veriliyorsa zamanlama kesintisizdir. Dağıtıcı bağlam değişimini yapar ve bunun süresi dağıtım gecikmesidir. Algoritmalar CPU kullanımı, verim, tamamlanma, bekleme ve yanıt süresine göre değerlendirilir. FCFS basittir ama konvoy etkisine açıktır. SJF ortalama beklemeyi azaltır ve patlamayı üstel ortalamayla tahmin eder. SRTF, SJF'nin kesintili hâlidir. Öncelikli zamanlamada düşük öncelikli süreçler açlığa düşebilir ve çözüm yaşlandırmadır. Hoca round robin ve çok seviyeli kuyrukları bir sonraki derse bıraktı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.